Cache Simulation System

Direct Mapping | LRU Replacement Policy | Analysis Writeup

Project Links

Resource	Link
Web App	https://simulationproject.onrender.com/
Walkthrough Video	(paste your video link here)
Source Code	https://github.com/DLSU-Garcia/CSC512C-Gamboa-Garcia-Simulation-Project

System Specifications

Parameter	Value
Cache Mapping Type	Direct Mapping
Replacement Policy	N/A(c/o Direct Mapping)
Cache Blocks (n)	4 blocks
Cache Line/Block Size	4 words per block
Main Memory Blocks	1024 blocks (fixed)
Read Policy	Non Load-Through
Cache Hit Time	1 ns
Main Memory Access Time	10 ns
Miss Penalty	1 ns (cache access) + 10ns x block size x 1 ns

Notes on Replacement Policy

In **Direct Mapping**, each memory block maps to exactly **one cache slot**, so there is never a choice of which block to evict — the incoming block always replaces whatever occupies its designated slot.

Address Decomposition

With 1024 memory blocks, 4 cache blocks, and 4 words per line, the address fields are derived as follows:

```
Memory blocks  = 1024 = 2^10  →  Block address field = 10 bits
Cache blocks   = 4 = 2^2   →  INDEX field         =  2 bits
Words per line =    4 = 2^2   →  WORD (offset) field =  2 bits
TAG field      = 10 - 4       →  TAG field        =  6 bits
Full address   = TAG[6] | INDEX[2] | WORD[2] = 10 bits total
```

Address Breakdown Example — Memory Block 5

Block 5 in binary: `0000000101` (10 bits)

Field	Bits	Bit Count	Derivation
TAG	000000	6	Tag = Main Memory Address Bits - block - word
BLOCK	01	2	Block = 2^k number of cache blocks
WORD	01	2	Word = 2^w Based on block size

Block 5 maps to **cache index 1** with tag **1**.

How Direct Mapping Works

In direct mapping, every memory block has **exactly one possible cache location**:

On a cache miss (non load-through policy):

- The entire block is fetched from main memory into the designated cache slot.
- The data is then served to the processor from the cache.
- The processor does NOT read directly from main memory (non load-through).

On a cache hit: Data is served directly from the cache in 1 ns.

Timing Model

Event	Time
Cache Hit	1 ns
Miss Penalty	1 ns (cache access) + 10ns x block size + 1 ns = 1ns +10ns x 4 +1ns = 42ns

Average Access Time	(Hit Rate × 1) + (Miss Penalty × Miss Rate) ns
Total Access Time	(# of blocks hit * block size) * read cache time + [read cache time + transfer lines to cache + transfer cache to cpu] x # of misses

Test Case A — Sequential Sequence

Sequence: `0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7`

Up to 2n = 8 blocks, repeated 2 times. Total accesses: 16.

Access Trace

1. Block 0: MISS - loaded into 0

2. Block 1: MISS - loaded into 1

3. Block 2: MISS - loaded into 2

4. Block 3: MISS - loaded into 3

5. Block 4: MISS - evicted block 0 from 0

6. Block 5: MISS - evicted block 1 from 1

7. Block 6: MISS - evicted block 2 from 2

8. Block 7: MISS - evicted block 3 from 3

9. Block 0: MISS - evicted block 4 from 0

10. Block 1: MISS - evicted block 5 from 1

11. Block 2: MISS - evicted block 6 from 2

12. Block 3: MISS - evicted block 7 from 3

13. Block 4: MISS - evicted block 0 from 0

14. Block 5: MISS - evicted block 1 from 1

15. Block 6: MISS - evicted block 2 from 2

16. Block 7: MISS - evicted block 3 from 3

Statistics

Metric	Value
Memory Access Count	16
Cache Hit Count	0
Cache Miss Count	16
Cache Hit Rate	0.00%
Cache Miss Rate	100.00%
Average Memory Access Time	(0 × 1) + (42.00 × 1) ns = 42.00 ns
Total Memory Access Time	(0 * 4) * 1ns + [1ns+ 1*4 + 10ns*4] x 16= 720 ns

Analysis

Test Case A demonstrates the **worst-case scenario** for direct mapping. Because the sequence spans `2n = 8` blocks with only `n = 4` cache slots, blocks 0–3 and blocks 4–7 **are assigned to the same cache indices**. Every time the second half of the sequence (blocks 4–7) is accessed, it evicts the first half (blocks 0–3). When the sequence repeats, blocks 0–3 must be re-loaded, immediately evicting 4–7 again. This pattern causes a **100% miss rate** with no benefit from caching whatsoever.

Test Case B — Mid-Repeat Blocks

Sequence: `0, 1, 2, 3, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 1, 2, 3, 4, 5, 6, 7`

Start at 0; repeat middle blocks 1–(n-1) twice; continue to 2n; repeat the entire sequence twice. Total accesses: 22.

Access Trace

1. Block 0: MISS - loaded into 0

2. Block 1: MISS - loaded into 1

3. Block 2: MISS - loaded into 2

4. Block 3: MISS - loaded into 3

5. Block 1: HIT (in cache 1)

6. Block 2: HIT (in cache 2)

7. Block 3: HIT (in cache 3)

8. Block 4: MISS - evicted block 0 from 0

9. Block 5: MISS - evicted block 1 from 1

10. Block 6: MISS - evicted block 2 from 2

11. Block 7: MISS - evicted block 3 from 3

12. Block 0: MISS - evicted block 4 from 0

13. Block 1: MISS - evicted block 5 from 1

14. Block 2: MISS - evicted block 6 from 2

15. Block 3: MISS - evicted block 7 from 3

16. Block 1: HIT (in cache 1)

17. Block 2: HIT (in cache 2)

18. Block 3: HIT (in cache 3)

19. Block 4: MISS - evicted block 0 from 0

20. Block 5: MISS - evicted block 1 from 1

21. Block 6: MISS - evicted block 2 from 2

22. Block 7: MISS - evicted block 3 from 3

Statistics

Metric	Value
Memory Access Count	22
Cache Hit Count	6
Cache Miss Count	16
Cache Hit Rate	27.3%

Cache Miss Rate	72.7%
Average Memory Access Time	(0.273 × 1) + (42.00 × 0.727) ns = 30.82 ns
Total Memory Access Time	(6 * 4) * 1ns + [1ns+ 1*4 + 10ns*4] x 16 = 744 ns

Analysis

Test Case B is the **best-performing** of the three. The mid-repeat subsequence (`1, 2, 3` accessed twice in a row) creates a **locality window** that the cache can exploit. Blocks 1, 2, and 3 are already in cache when they are accessed the second time within the same pass, generating **3 hits per repetition** (6 hits total across 2 passes). The sections that span `2n` blocks still cause conflicts identical to Test A, contributing the 16 unavoidable misses. The 27.27% hit rate represents the only scenario in these test cases where direct mapping provides measurable benefit.

Test Case C — Random Sequence

Sequence (64 unique blocks):

1. Block 984: MISS - loaded into 0
2. Block 94: MISS - loaded into 2
3. Block 7: MISS - loaded into 3
4. Block 931: MISS - evicted block 7 from 3
5. Block 983: MISS - evicted block 931 from 3
6. Block 498: MISS - evicted block 94 from 2
7. Block 56: MISS - evicted block 984 from 0
8. Block 809: MISS - loaded into 1
9. Block 1020: MISS - evicted block 56 from 0
10. Block 492: MISS - evicted block 1020 from 0
11. Block 34: MISS - evicted block 498 from 2
12. Block 562: MISS - evicted block 34 from 2
13. Block 394: MISS - evicted block 562 from 2
14. Block 107: MISS - evicted block 983 from 3
15. Block 72: MISS - evicted block 492 from 0
16. Block 628: MISS - evicted block 72 from 0
17. Block 626: MISS - evicted block 394 from 2
18. Block 511: MISS - evicted block 107 from 3

19. Block 278: MISS - evicted block 626 from 2
20. Block 421: MISS - evicted block 809 from 1
21. Block 797: MISS - evicted block 421 from 1
22. Block 547: MISS - evicted block 511 from 3
23. Block 610: MISS - evicted block 278 from 2
24. Block 462: MISS - evicted block 610 from 2
25. Block 403: MISS - evicted block 547 from 3
26. Block 427: MISS - evicted block 403 from 3
27. Block 444: MISS - evicted block 628 from 0
28. Block 579: MISS - evicted block 427 from 3
29. Block 859: MISS - evicted block 579 from 3
30. Block 668: MISS - evicted block 444 from 0
31. Block 176: MISS - evicted block 668 from 0
32. Block 931: MISS - evicted block 859 from 3
33. Block 647: MISS - evicted block 931 from 3
34. Block 1010: MISS - evicted block 462 from 2
35. Block 575: MISS - evicted block 647 from 3
36. Block 936: MISS - evicted block 176 from 0
37. Block 243: MISS - evicted block 575 from 3
38. Block 270: MISS - evicted block 1010 from 2
39. Block 29: MISS - evicted block 797 from 1
40. Block 304: MISS - evicted block 936 from 0
41. Block 133: MISS - evicted block 29 from 1
42. Block 755: MISS - evicted block 243 from 3
43. Block 728: MISS - evicted block 304 from 0
44. Block 182: MISS - evicted block 270 from 2
45. Block 999: MISS - evicted block 755 from 3
46. Block 103: MISS - evicted block 999 from 3
47. Block 859: MISS - evicted block 103 from 3
48. Block 733: MISS - evicted block 133 from 1
49. Block 869: MISS - evicted block 733 from 1
50. Block 569: MISS - evicted block 869 from 1
51. Block 475: MISS - evicted block 859 from 3
52. Block 346: MISS - evicted block 182 from 2
53. Block 890: MISS - evicted block 346 from 2
54. Block 407: MISS - evicted block 475 from 3
55. Block 988: MISS - evicted block 728 from 0

56. Block 569: HIT (in cache 1)
57. Block 289: MISS - evicted block 569 from 1
58. Block 188: MISS - evicted block 988 from 0
59. Block 88: MISS - evicted block 188 from 0
60. Block 263: MISS - evicted block 407 from 3
61. Block 993: MISS - evicted block 289 from 1
62. Block 833: MISS - evicted block 993 from 1
63. Block 345: MISS - evicted block 833 from 1
64. Block 765: MISS - evicted block 345 from 1

Statistics

Metric	Value
Memory Access Count	64
Cache Hit Count	1
Cache Miss Count	63
Cache Hit Rate	1.56%
Cache Miss Rate	98.44%
Average Memory Access Time	(0.0156 × 1) + (42.00 × 0.9844) ns = 41.36 ns
Total Memory Access Time	(1 * 4) * 1ns + [1ns+ 1*4 + 10ns*4] x 63 = 2839 ns

Analysis

The random sequence accesses 64 memory blocks with **only 1 cache hit**. There is only one 1 cache hit from a lucky instance of having block 569 have an available cache block that maps to it. With only 4 cache slots and 63 unique block accesses, temporal locality could not be exploited well due to the randomness of the accesses and the relative big size difference of the main memory to the cache memory. This result is expected and is not a flaw in the cache design.

Comparative Analysis

Metric	Test A (Sequential)	Test B (Mid-Repeat)	Test C (Random)
Total Accesses	16	22	64

Hits	0	6	0
Misses	16	16	64
Hit Rate	0.00%	27.27%	0.00%
Miss Rate	100.00%	72.73%	100.00%
Avg Access Time	42.00 ns	30.82 ns	41.36 ns
Total Access Time	720 ns	744 ns	2839 ns

Key Observations

- Direct Mapping did not perform at all when data is retrieved sequentially from the main memory since each main memory block is mapped to cache slots that are too far apart.
- Direct Mapping did get to have some hits when data is retrieved mid-repeat, since mid-repeat benefits from spatial-locality that Direct Mapping can provide.

Application Features

- Configurable Parameters — Cache blocks (min 4, power of 2) and words per line (min 2, power of 2) are user-adjustable inputs.
- Step-by-Step Animated Tracing — Users can step through each memory access with visual highlighting of which cache slot is being read or written.
- Final Snapshot Mode — Option to skip directly to the final cache state after all accesses.
- Text Log — A full access-by-access text log is always generated showing block number, cache index, tag, hit/miss result, and cache state after each access.
- Statistics Panel — Displays all required output metrics: memory access count, hit count, miss count, hit rate, miss rate, average access time, and total access time.
- Three Built-in Test Cases — Sequential, Mid-Repeat, and Random sequences are pre-loaded and selectable from the interface.

Full Specifications Summary

Parameter	Specification
Cache Mapping	Direct Mapping
Replacement Policy	LRU (functionally direct eviction in direct mapping)
Cache Line Size	4 words (configurable, min 2, power of 2)

Cache Blocks	4 blocks (configurable, min 4, power of 2)
Memory Blocks	1024 blocks (fixed)
Read Policy	Non Load-Through
Cache Hit Time	1 ns
Memory Access Time	10 ns
Address Format	Full address = TAG[6] | INDEX[2] | WORD[2] = 10 bits total
Programming Stack	Python, NiceGUI
Web Platform	Render